

II2520 - Einführung in Deep Learning

Vorlesung 9: Natural Language Processing

Recap

Attention-Mechanismus

- $5' / \text{Ü} \alpha' ' :$
 - Relevante Informationen aus dem gesamten Input dynamisch gewichten.
 - Fokus auf Schlüsselstellen im Kontext, unabhängig von deren Position.
- $^{\circ} \text{t} \cdot \text{U} \grave{\text{a}} \text{Ö} \cdot \grave{\text{a}} \text{Ü}' \text{Ü}' \text{Ü} :$
 - $\neq ' / \neq :$ Was suche ich?
 - $5' \quad 5$ Wo finde ich es?
 - $\text{Y} \text{t} \text{Ä}' \quad \text{Y}$ Welche Information ist relevant?
 - $\text{A}' \text{Ä} ! \quad ' \text{Ü} \grave{\text{a}} \text{Ü}$ Vergleich jedes Tokens mit allen anderen im Input.

Transformer-Architektur

- $\infty' \quad \grave{\text{a}} \text{Ä} \quad \grave{\text{a}} \text{Ü} \text{d} \text{Ü}^6 4] :$
Ermöglicht paralleles Training und Verarbeitung langer Sequenzen.
- $^{\circ} \text{t} \cdot \zeta' \quad \text{t} \text{Ü} \alpha' \quad \text{c} \text{Ä} :$
 - $\sim \text{Ü} \grave{\text{a}} \alpha' / :$ Analysiert Eingabetext und erstellt Kontextrepräsentationen.
 - $s' \quad \text{v} \grave{\text{a}} \alpha' / :$ Generiert Ausgabesequenzen (z. B. Übersetzung).
- $\text{Y} \grave{\text{a}} / \quad ' \text{c} \text{Ä} :$
 - Kein sequentielles Processing (wie bei RNNs/LSTMs).
 - Skalierbar für große Datenmengen.

Inhalt der Vorlesung

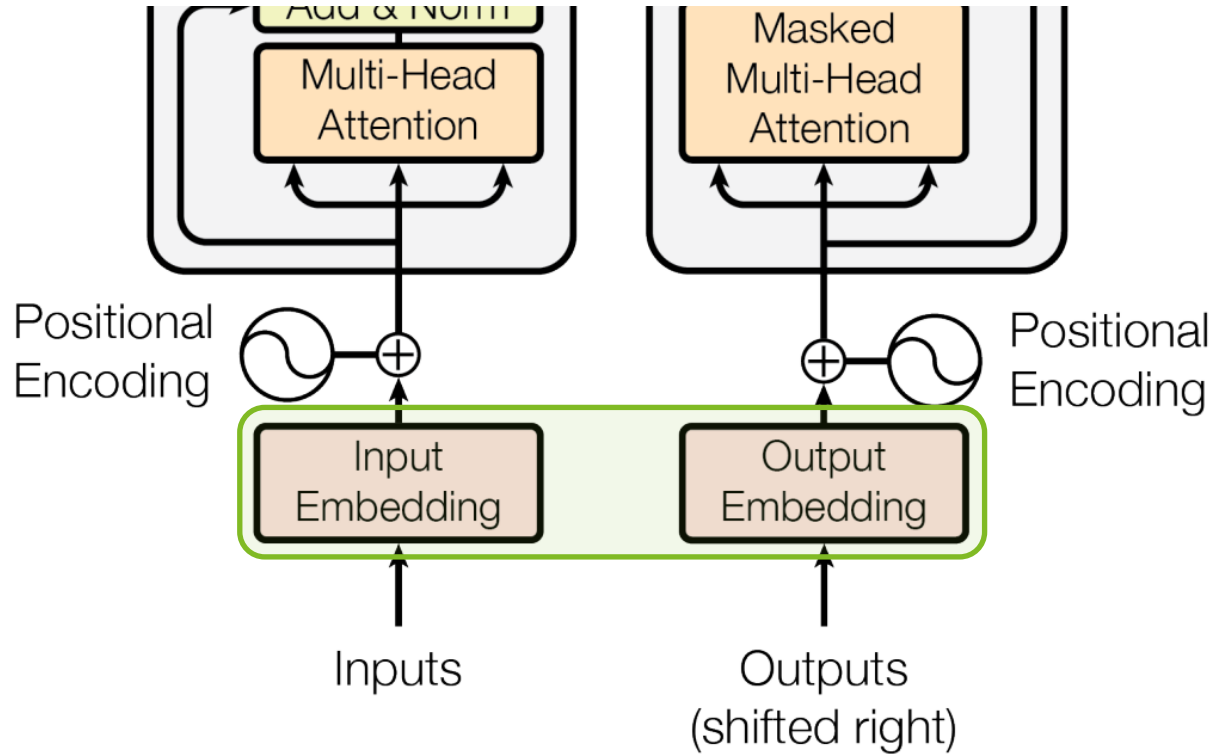
- Tokenizer
"Worte in Einheiten zerlegen –
die Grundlage jeder NLP-Reise."
- Word-Embeddings (Pretraining)
"Von Wörtern zu Zahlen – semantische
Bedeutungen in Vektoren fassen."



Lernziele

1. Grundlegendes Verständnis entwickeln
Sie können erklären, was ein Tokenizer ist und wie er Text in verarbeitbare Einheiten zerlegt.
2. Vergleich verschiedener Tokenizer-Methoden
Sie können verschiedene Arten von Tokenizern (z. B. Word-, Subword-, und Character-Level) beschreiben und ihre Vor- und Nachteile benennen.
3. Funktionsweise von Word Embeddings verstehen
Sie können die Grundprinzipien von Word Embeddings erklären und die Unterschiede zwischen One-Hot-Encoding und Word2Vec erläutern.
4. Zusammenhang zwischen Tokenizer und Word Embeddings erkennen
Sie können die Rolle des Tokenizers bei der Vorbereitung von Textdaten und die Verwendung von Word Embeddings in NLP-Pipelines beschreiben.
5. Praktische Relevanz in modernen NLP-Modellen erkennen
Sie können die Bedeutung von Tokenizer und Word Embeddings in aktuellen NLP-Modellen wie Transformer, BERT oder GPT erklären.

Wie können wir Wörter in Zahlen übersetzen?



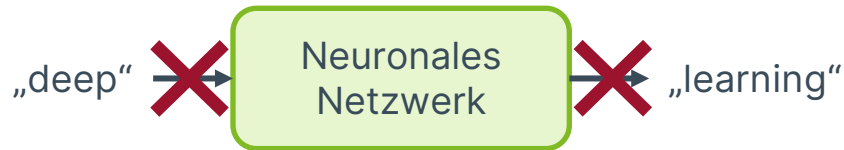
Welches Wort kommt als nächstes?

Jeden Morgen lauf ich langsam zur Uni.

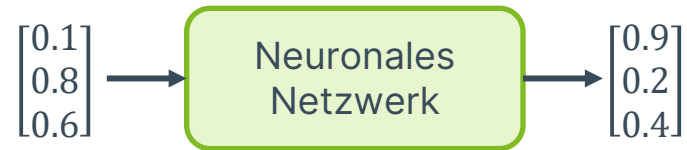
Basierend auf diesen Wörtern

Sag das nächste
Wort voraus

Repräsentation von Sprach für ein Neuronales Netzwerk



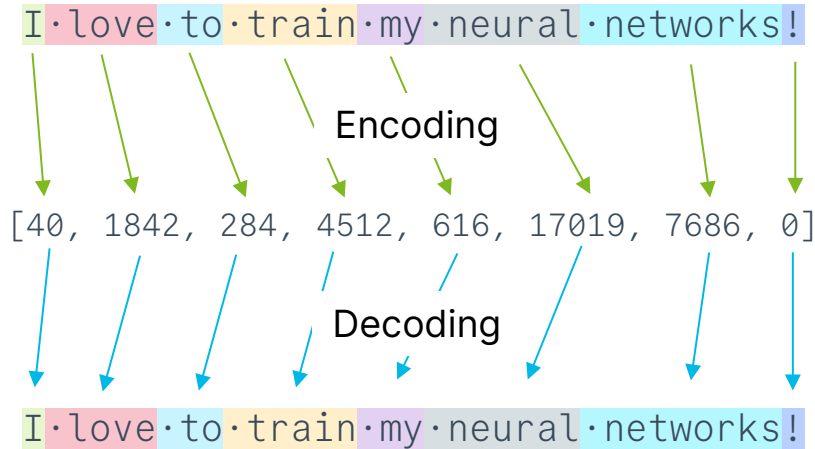
Neuronale Netze können U d Wörter interpretieren



Neuronale Netze ζ/t v IO Ű numerischen Input

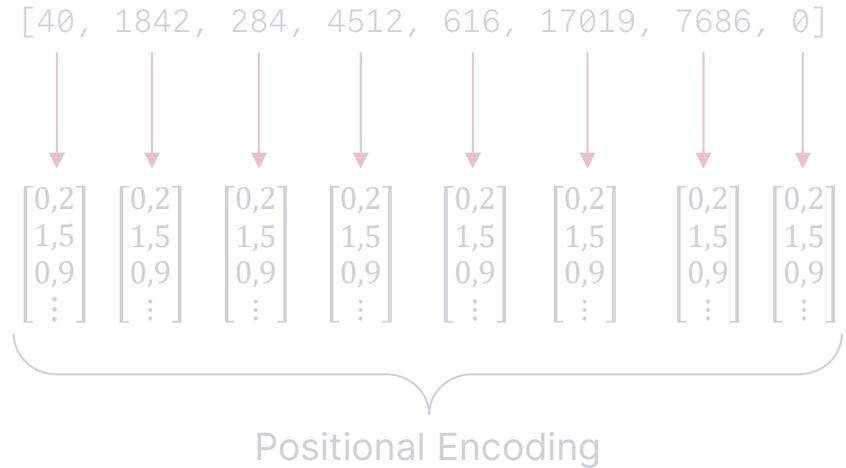
Effiziente Darstellung von Wörtern für NN

1. Tokenizer



Zerlegt den Eingabetext in kleinere Einheiten (Tokens)

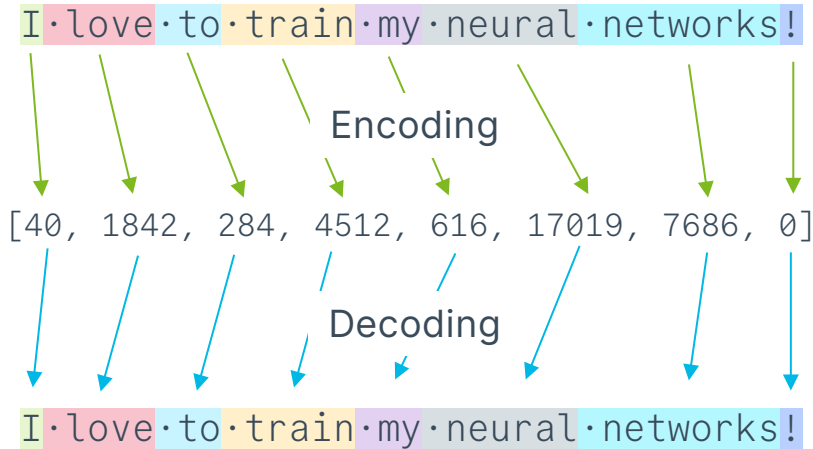
2. Embedding (Pretraining)



Anschließende Umwandlung in Vektor-Repräsentationen (Embeddings)

Tokenizer

Was macht ein Tokenizer?



- Ein Vorverarbeitungswerkzeug, das Text in strukturierte Einheiten, sogenannte „Token“, zerlegt.
- Konvertiert Zeichenketten in sinnvolle Segmente (z. B. Wörter, Teilwörter, Zeichen).
- Stellt sicher, dass Sprachmodelle strukturierten, interpretierbaren Text anstelle von Rohdaten verarbeiten.
- Unverzichtbar, damit die Modelle natürliche Sprache verstehen und effektiv damit arbeiten können.

Kurze Geschichte



Geht es auch ohne?

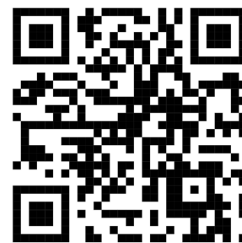
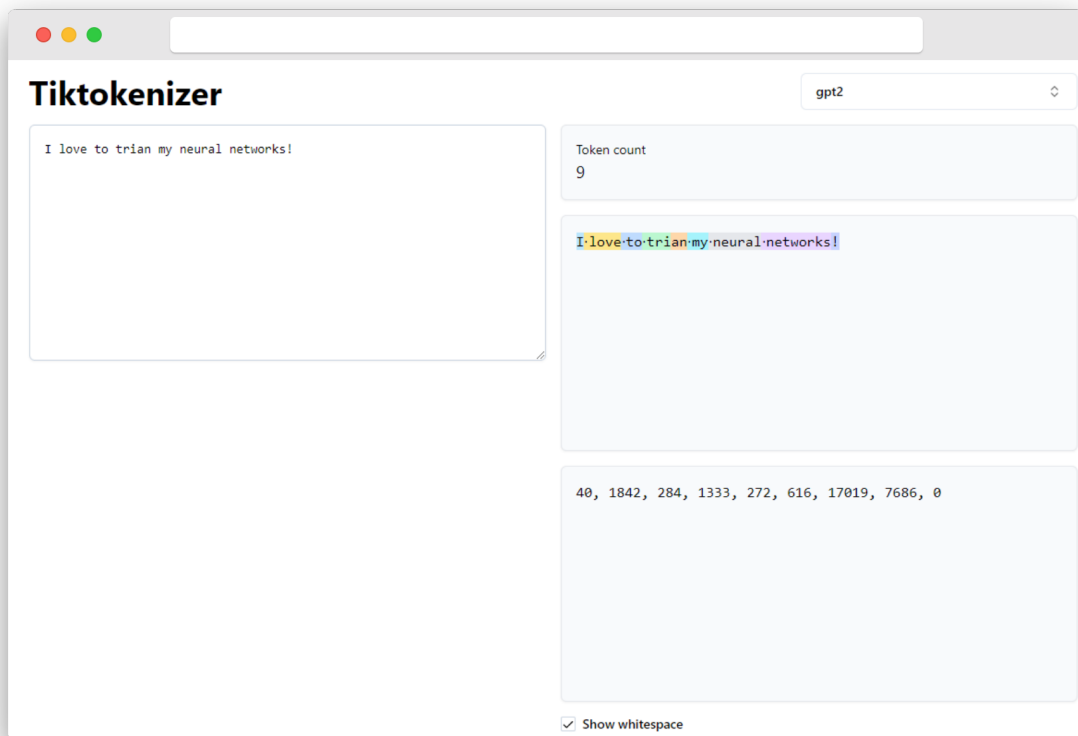
“Much glory awaits someone who can delete the need for tokenization. But meanwhile, let's learn about it.”

Andrej Karpathy

D. Choe, R. Al-Rfou, M. Guo, H. Lee, and N. Constant, "[Bridging the Gap for Token-Free Language Models](#)," arXiv, Jan. 2019

Pagnoni, A., et al. "[Byte Latent Transformer: Patches Scale Better Than Tokens](#)," on Meta, Dec. 2024.

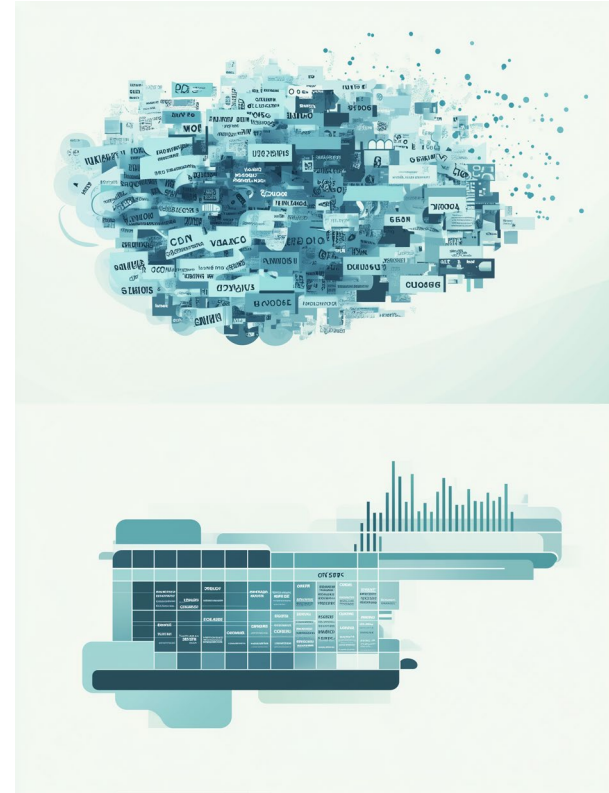
Beispiel: Verschiedene Tokenizer



tiktokenizer.vercel.app

Byte Pair Encoding (BPE)

- Problem mit Tokenisierung auf Wortebene:
 - Große Vokabulargrößen
 - Viele Wörter außerhalb des Vokabulars
 - Spärliche Repräsentationen und Ineffizienz
- BPE als Lösung:
 - Erlernt eine überschaubare Menge von Teilworten
 - Gleichgewicht zwischen Wortschatzgröße und Flexibilität der Darstellung



BPE – Algorithmus

1. Schritt: Aufteilen von Text in einzelne Wörter → Corpus

```
1 function BytePairEncoding(corpus, k):
2     # Initialize V to all unique characters in corpus
3     V ← unique_characters(corpus)
4
5     for i in 1 to k:
6         # Count frequencies of all adjacent pairs in the corpus
7         pair_freqs ← compute_pair_frequencies(corpus)
8
9         # Identify the pair (tL, tR) with the highest frequency
10        (tL, tR) ← pair_with_max_frequency(pair_freqs)
11
12        # Create a new token by merging the pair
13        t_new ← tL + tR
14
15        # Add the new token to the vocabulary
16        V ← V ∪ {t_new}
17
18        # Replace every occurrence of (tL, tR) in the corpus with t_new
19        corpus ← merge_pairs_in_corpus(corpus, tL, tR, t_new)
20
21    return V
```

R. Sennrich, B. Haddow, and A. Birch, "[Neural Machine Translation of Rare Words with Subword Units](#),"
Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Jan. 2016

Demo Tokenization

Iteration	Corpus	Vocabulary
0	AACGCACTATATA	{A,T,C,G}
1	A A C G C A C TA TA TA	{A,T,C,G,TA}
2	A AC G C AC TA TA TA	{A,T,C,G,TA,AC}
3	A AC G C AC TA TA TA	...

BPE – Pros & Cons

Vorteile

- ✓ Kompaktes Vokabular
- ✓ Gute Verarbeitung auch für seltene Wörter
- ✓ Teilweise Unabhängig von Sprache

Nachteile

- ✗ Immer noch willkürliche Aufteilungen
- ✗ Auswahl der Vokabular Größe
- ✗ Nicht kontextabhängig
- ✗ Nur teilweise Unabhängig von Sprache
→ BPE nimmt an, dass Wörter durch Leerzeichen getrennt sind

Geht Besser: z.B. WordPiece oder SentencePiece

Anzahl Tokens = Geld

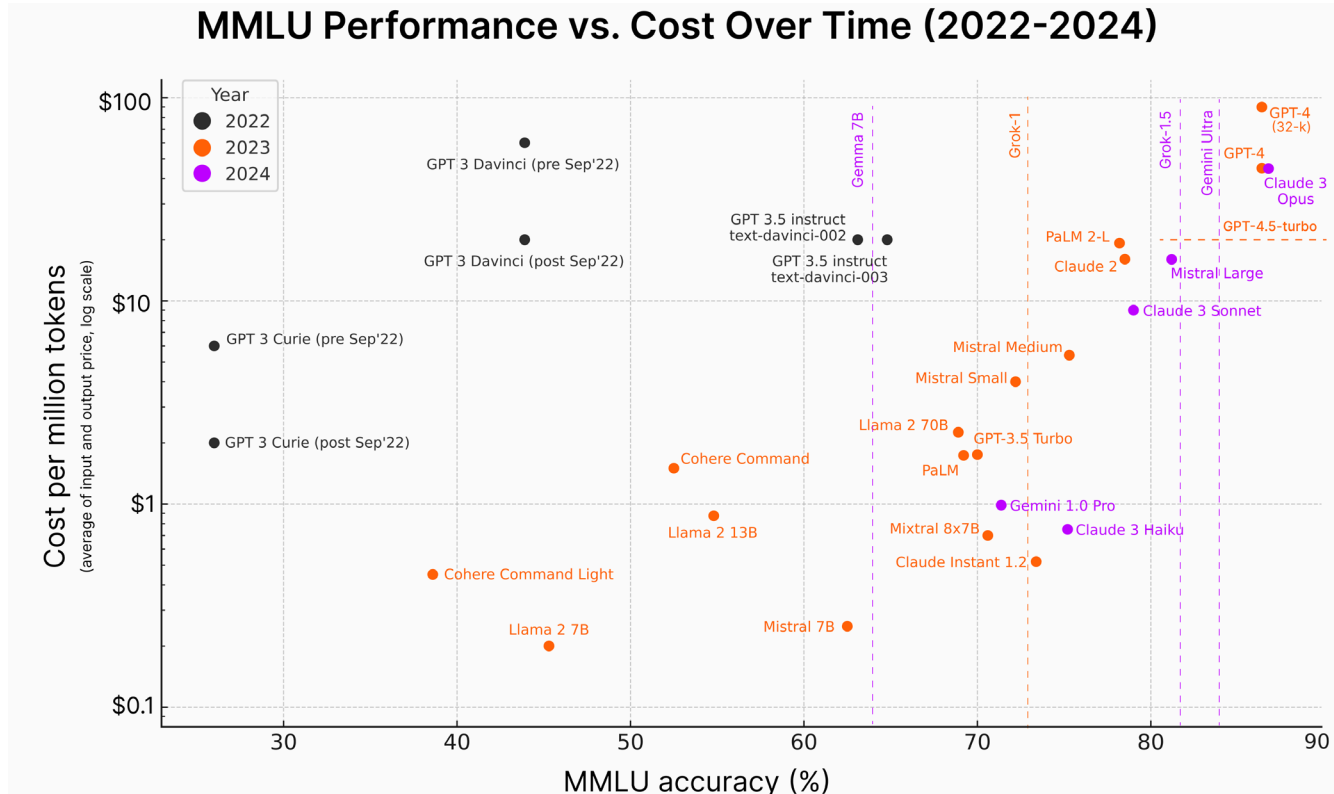


Bild-Quelle: <https://semaphore.substack.com/p/the-cost-of-reasoning-in-raw-intelligence>

Limitierung durch Tokenizer

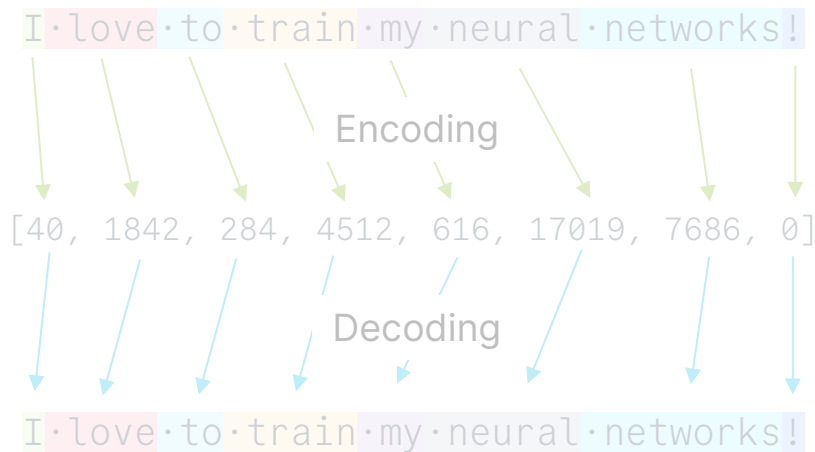
- Why can't LLM spell words? **Tokenization.**
- Why can't LLM do super simple string processing tasks like reversing a string? **Tokenization.**
- Why is LLM worse at non-English languages (e.g. Japanese)? **Tokenization.**
- Why is LLM bad at simple arithmetic? **Tokenization.**
- Why did GPT-2 have more than necessary trouble coding in Python? **Tokenization.**
- Why did my LLM abruptly halt when it sees the string "<|endoftext|>"? **Tokenization.**
- What is this weird warning I get about a "trailing whitespace"? **Tokenization.**
- Why should I prefer to use YAML over JSON with LLMs? **Tokenization.**
- Why is LLM not actually end-to-end language modeling? **Tokenization.**
- What is the real root of suffering? **Tokenization.**

Quelle: Andrej Karpathy – [Let's build the GPT Tokenizer](#)

Embedding (Pretraining)

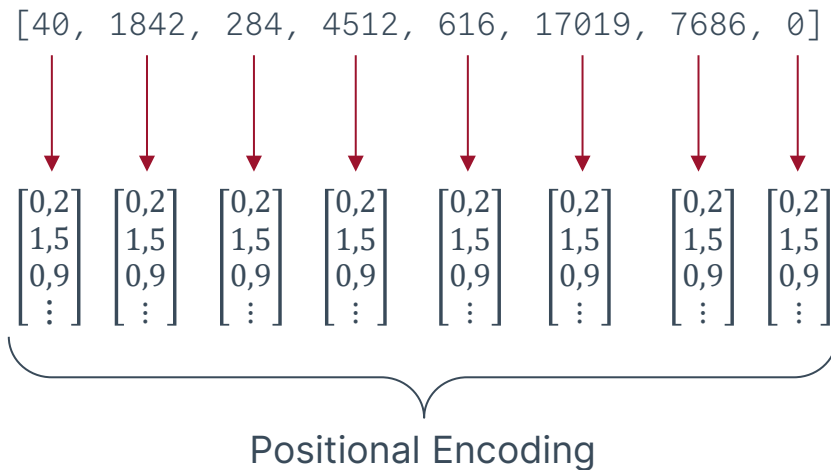
Effiziente Darstellung von Wörtern für NN

1. Tokenizer



Zerlegt den Eingabetext in kleinere Einheiten (Tokens)

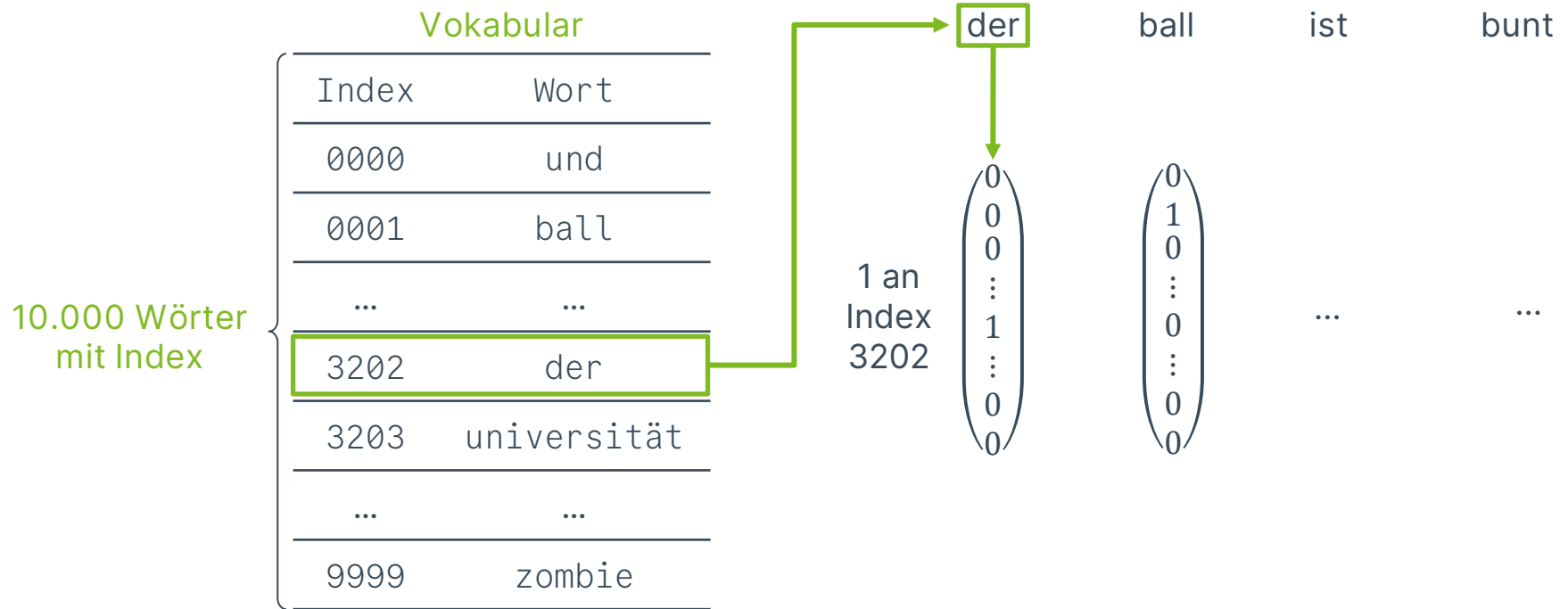
2. Embedding (Pretraining)



Anschließende Umwandlung in Vektor-Repräsentationen (Embeddings)

Embedding

- Idee: One-Hot-Encoding

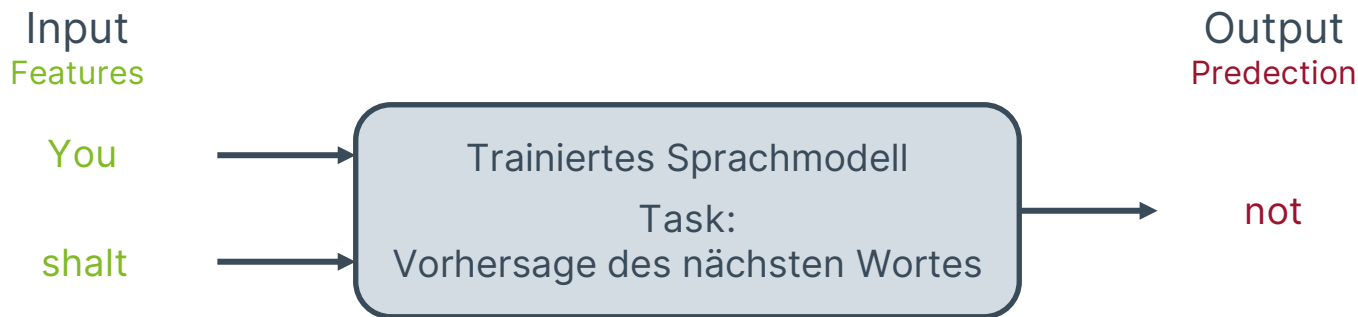


Embedding

- Die Probleme mit **Sparse-One-Hot-Codierungen**:
 - **Das Problem der Ähnlichkeit**: Idealerweise würden wir wollen, dass ähnliche Wörter wie "Katze" und "Tiger" einigermaßen ähnliche Eigenschaften haben. Aber mit diesen One-Hot-Vektoren ist "Katze" so ähnlich zu "Tiger" wie zu jedem anderen Wort!
 - **Das Problem der Wortschatzgröße**: Wenn Sie bei diesem Ansatz Ihr Vokabular um n vergrößern, werden auch die Vektoren für die Merkmalsgröße um die Länge n größer.
 - **Das Problem der Berechnung**: Die One-Hot-Vektoren enthalten überwiegend Nullen, was zu einer sehr hohen Dimensionalität und sparsamen Besetzung führt. Solche Merkmale stellen viele maschinelle Lernmodelle vor Herausforderungen und beeinträchtigen deren Leistungsfähigkeit.
- Lösung: dichte, semantisch aussagekräftige Merkmalsvektoren

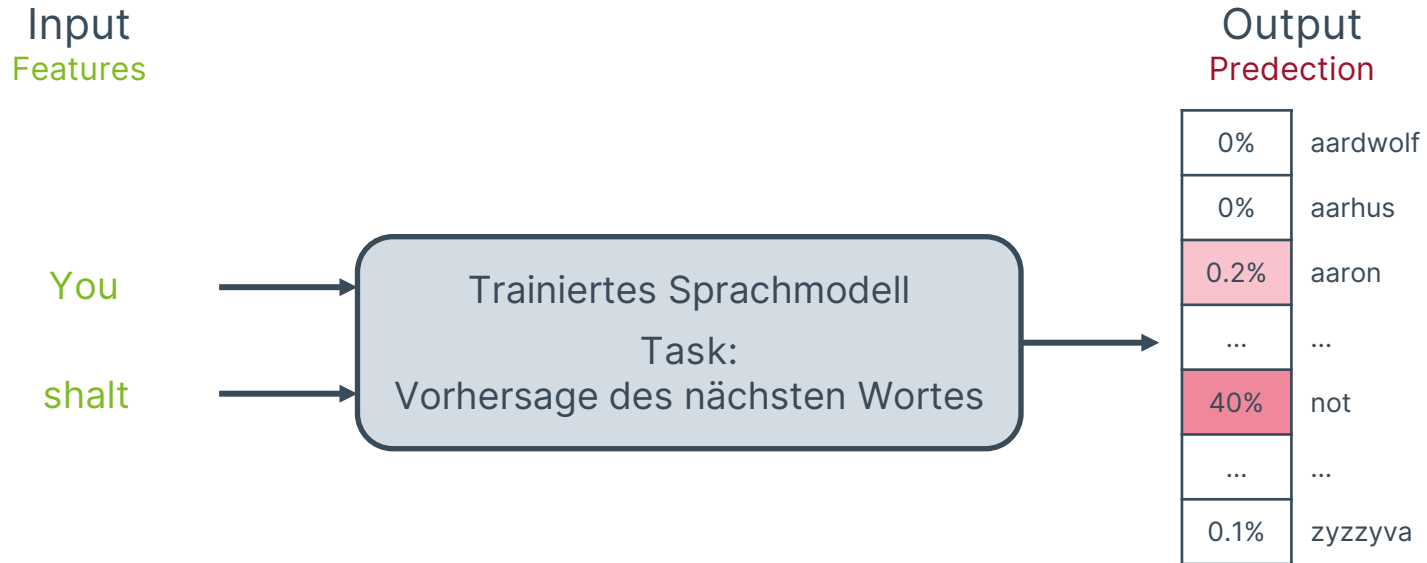
Sprachmodell als Lösung für Embedding Vektoren

- Was ist ein Sprachmodell?
 - Ein System, das die Wahrscheinlichkeit von Wörtern in einer Sequenz vorhersagt.
- Next-Word Prediction ist eine der prominentesten Anwendungen der natürlichen Sprachverarbeitung (NLP).
- Beispiel: Zwei vorherige Wörter ("thou, shalt"):
 - Das Modell schlägt das wahrscheinlichste nächste Wort vor (im Beispiel: "not").



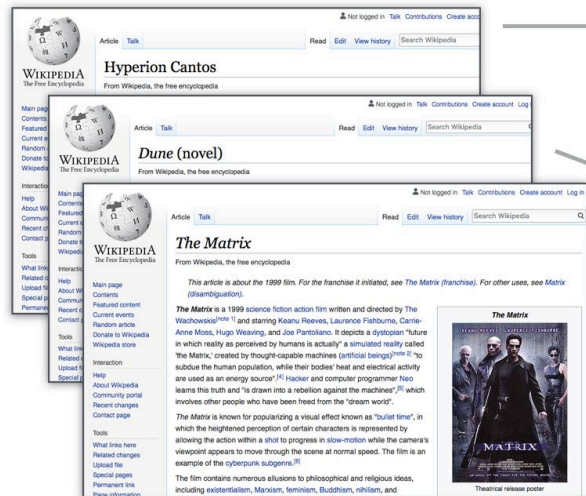
Sprachmodell als Lösung für Embedding Vektoren

- In der Praxis gibt das Modell jedoch nicht nur ein Wort aus. Es gibt vielmehr eine Wahrscheinlichkeitsbewertung für alle Wörter aus, die es kennt (das "Vokabular" des Modells, das von einigen tausend bis zu über einer Million Wörtern reichen kann).



Sprachmodell-Training

- Wir erhalten eine große Menge an Textdaten (z. B. alle Wikipedia-Artikel).
- Wir haben ein Fenster (z. B. mit drei Wörtern), das wir über den gesamten Text schieben.
- Das gleitende Fenster erzeugt Trainingsmuster für unser Modell



The **Hyperion Cantos** is a series of science fiction novels by Dan Simmons. The title *The Fall of Hyperion*^[1] and later came to refer to the overall story, including *Endymion*, *The Rise of Endymion*, and a number of short stories.^[2] More narrowly, inside the fictional storyline, after the first volume, the Hyperion Cantos is an epic poem written by the character Martin Silenus covering in verse form the events of the first book.^[3]

Thou shalt not **make a machine** in the likeness of a human mind

Sliding window across running text

Dataset

								Input 1	Input 2	Output
thou	shalt	not	make	a	machine	in	the ...	thou	shalt	not
thou	shalt	not	make	a	machine	in	the ...	shalt	not	make
thou	shalt	not	make	a	machine	in	the ...	not	make	a
thou	shalt	not	make	a	machine	in	the ...	make	a	machine
thou	shalt	not	make	a	machine	in	the ...	a	machine	in

In der Praxis werden die Modelle in der Regel trainiert, während wir das Fenster verschieben.

Quelle: [The Illustrated Word2Vec – Jay Alamar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://github.com/jalammar/the-illustrated-word2vec)

N-Grams

Ein Fenster der Größe N wird durch den Text geschoben und bildet einen Datensatz zur Vorhersage des letzten Wortes

Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...

Dataset

Input 1	Input 2	Output
thou	shalt	not
shalt	not	make
not	make	a
make	a	machine
a	machine	in

Quelle: [The Illustrated Word2vec – Jay Alammar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://github.com/jalammar/word2vec-illustrated)

Historie = genug Kontext?

Welches Wort kommt in die Lücke?

Ich trinke aus einer _____

Welches Wort kommt in die Lücke?

Ich trinke aus einer _____ Tasse.

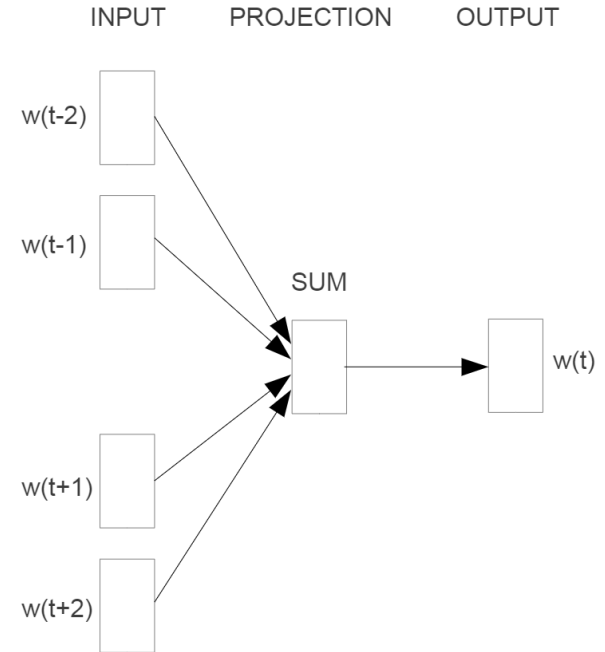
Continuous Bag of Words

Ich trinke aus einer _____ Tasse und

aus	einer	großen	Tasse	und
-----	-------	--------	-------	-----

Dataset

Input 1	Input 2	Input 3	Input 4	Output
aus	einer	Tasse	und	großen

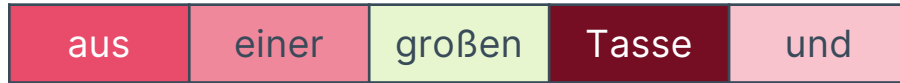


CBOW

Mikolov, T., Chen, K., Corrado, G.S., & Dean, J. (2013). [Efficient Estimation of Word Representations in Vector Space](#). *International Conference on Learning Representations*.

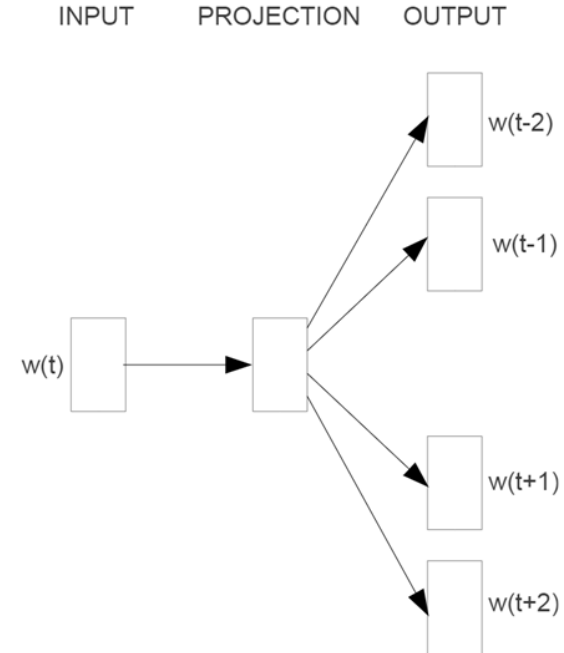
Skip Gram

Ich trinke aus einer großen Tasse und



Dataset

Input	Output
großen	aus
großen	einer
großen	Tasse
großen	und

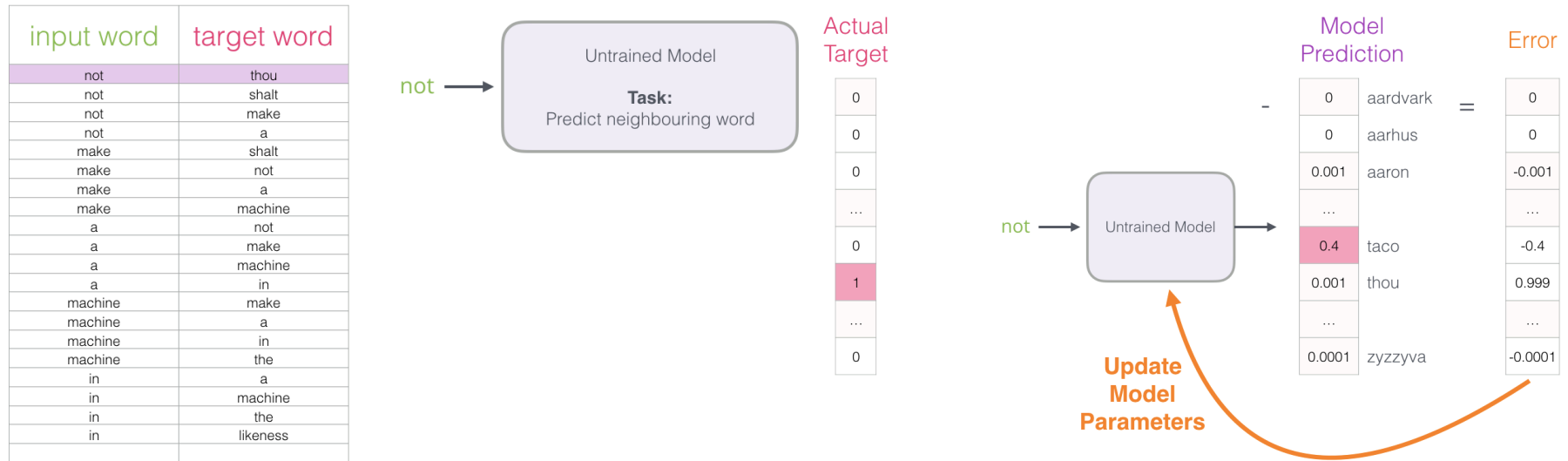


Skip-gram

Mikolov, T., Chen, K., Corrado, G.S., & Dean, J. (2013). [Efficient Estimation of Word Representations in Vector Space](#). *International Conference on Learning Representations*.

Trainingsprozess

Nachdem wir unseren Trainingsdatensatz durch Skip-Gram generiert haben, verwenden wir ihn, um ein grundlegendes neuronales Sprachmodell zu trainieren:



Quelle: [The Illustrated Word2vec – Jay Alamar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://alammar.github.io)

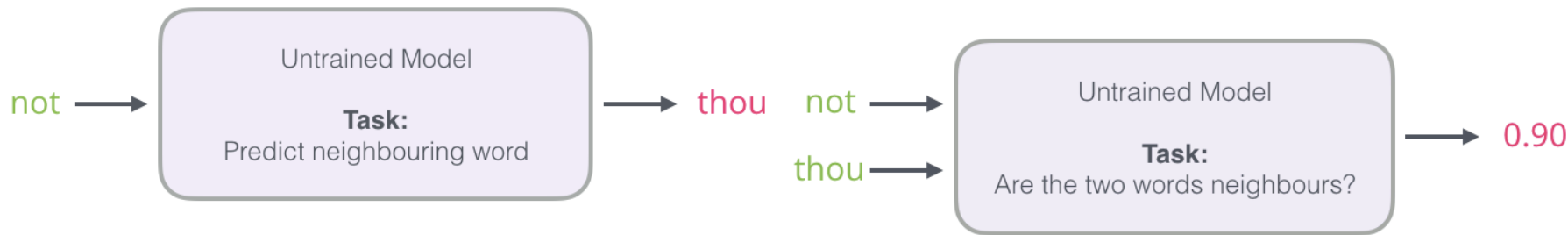
Word-Embeddings

Um mit einem leistungsstarken Modell qualitativ hochwertige Embeddings zu erzeugen, können wir die Aufgabe des Modells von der Vorhersage eines benachbarten Wortes auf ein anderes umstellen:

- Wechseln zu einem Modell, das das Eingabe- und das Ausgabewort nimmt und einen Wert ausgibt, der angibt, ob sie Nachbarn sind oder nicht (0 für "nicht benachbart", 1 für "benachbart").

Change Task from

To:



Quelle: [The Illustrated Word2vec – Jay Alammar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://jalammar.github.io/visualizing-machine-learning-one-concept-at-a-time/)

Datensatz erweitern

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1

Quelle: [The Illustrated Word2vec – Jay Alammar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://alammar.github.io/)

Contrastive Learning

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	make	1

Pick randomly from vocabulary
(random sampling)

Word	Count	Probability
aardvark		
aarhus		
aaron		
taco		
thou		
zyzzyva		

not →

thou →

Smartass Model

Task:

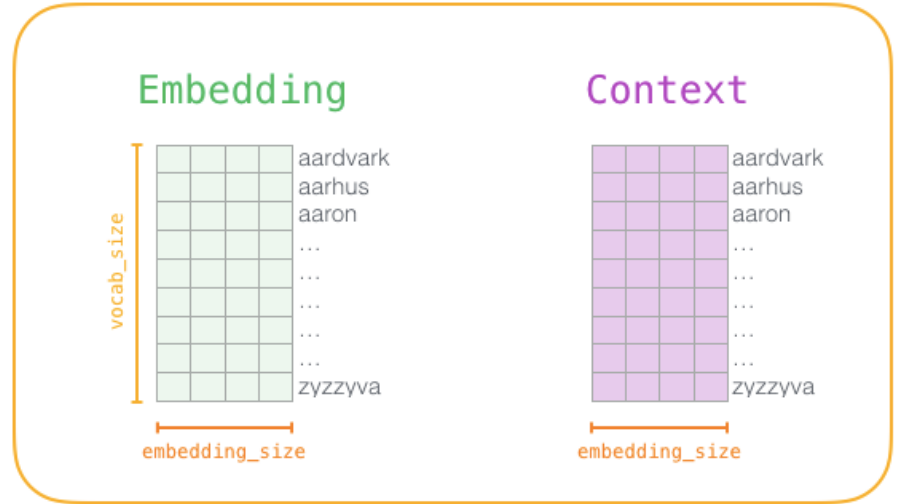
Are the two words neighbours?

```
def model(in, out):
    return 1.0
```

Quelle: [The Illustrated Word2vec – Jay Alamar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://alammar.github.io/illustrated-word2vec/)

Word2vec Training

- Vor dem Start des Trainingsprozesses bereiten wir den Text vor, gegen den das Modell trainiert wird. Dabei legen wir die Größe des Vokabulars fest (vocab_size, z.B. 10.000) und bestimmen, welche Wörter dazu gehören.
- Zu Beginn des Trainings erstellen wir zwei Matrizen - eine Embedding-Matrix und eine Context-Matrix. Diese Matrizen enthalten für jedes Wort im Vokabular ein Embedding, wobei die zweite Dimension die Länge der Embeddings angibt (üblich embedding_size, z.B. 300).



Quelle: [The Illustrated Word2vec – Jay Alamar – Visualizing machine learning one concept at a time. \(jalamar.github.io\)](https://alammar.github.io/illustrated-word2vec/)

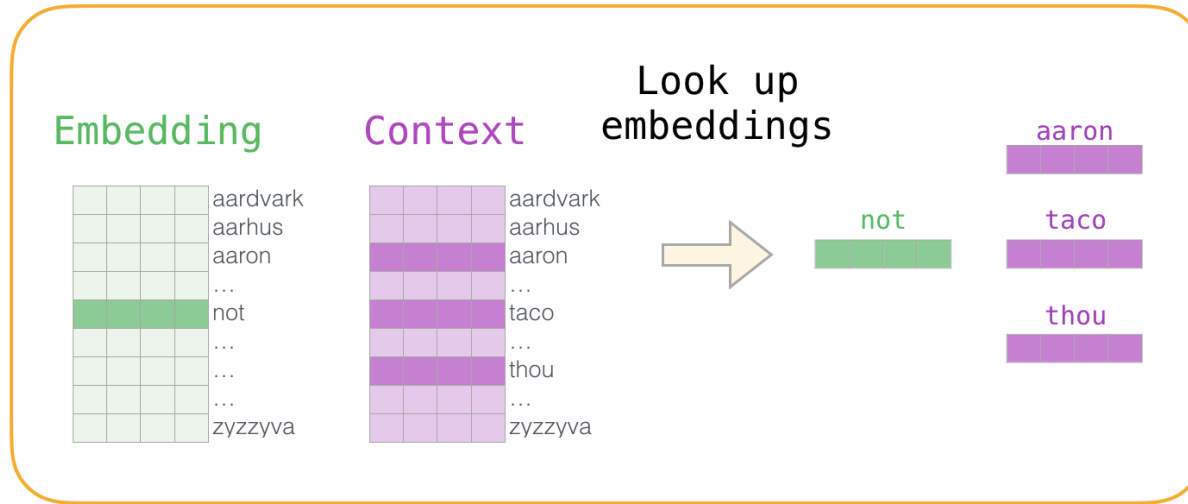
Word2vec Training

1. Initialisierung der Matrizen:

- Zu Beginn werden Embedding- und Context-Matrizen mit zufälligen Werten initialisiert.

2. Trainingsschritt:

- Auswahl eines positiven Beispiels und zugehöriger negativer Beispiele.
- Einbettungen für Input- und Kontextwörter werden aus den Matrizen abgerufen.



Quelle: [The Illustrated Word2vec – Jay Alammar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://alammar.github.io/Visualizing-machine-learning-one-concept-at-a-time/)

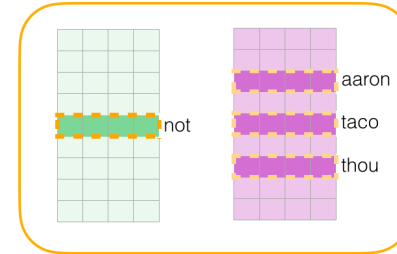
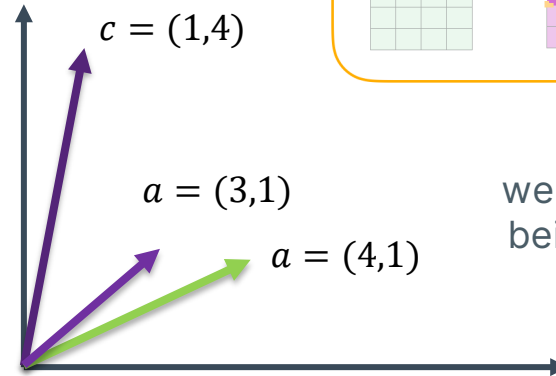
Word2vec Training

3. Ähnlichkeitsberechnung:

- Berechnung des Skalarprodukts zwischen Input- und Kontexteinbettungen.
- Anwendung der Sigmoid-Funktion zur Umwandlung der Ähnlichkeitswerte in Wahrscheinlichkeiten.

input word	output word	target	input • output	sigmoid()	Error
not	thou	1	0.2	0.55	0.45
not	aaron	0	-1.11	0.25	-0.25
not	taco	0	0.74	0.68	-0.68

4. Update der Modell-Parameter

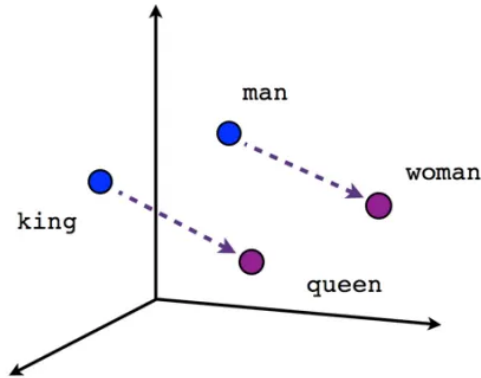


Update
Model
Parameters

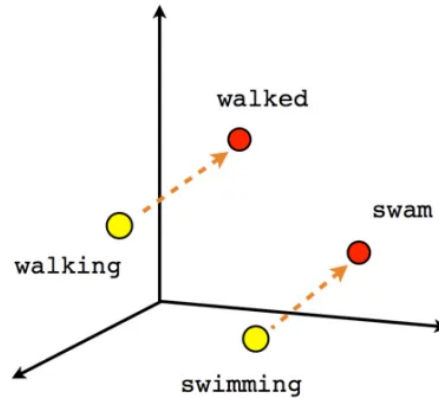
wenn zwei Vektoren nahe
beieinander liegen, ist ihr
Skalarprodukt groß

Quelle: [The Illustrated Word2vec – Jay Alamar – Visualizing machine learning one concept at a time. \(jalamar.github.io\)](https://github.com/jalammar/visualizing-machine-learning-one-concept-at-a-time)

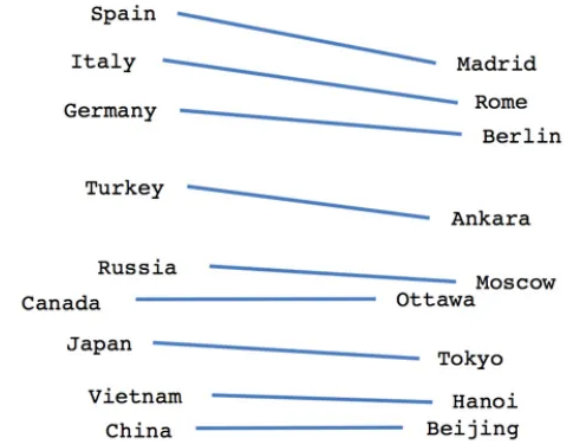
Word2Vec Demo



Male-Female



Verb tense



Country-Capital

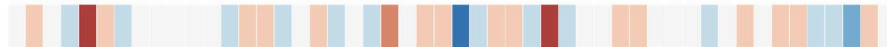
Bild-Quelle: [Towards Data Science Blog](https://towardsdatascience.com/word-embeddings-visualized-3d-plot)

Word2Vec Demo

“king”



“Man”



“Woman”



king - man + woman $\approx$ queen

king



man



woman



king-man+woman

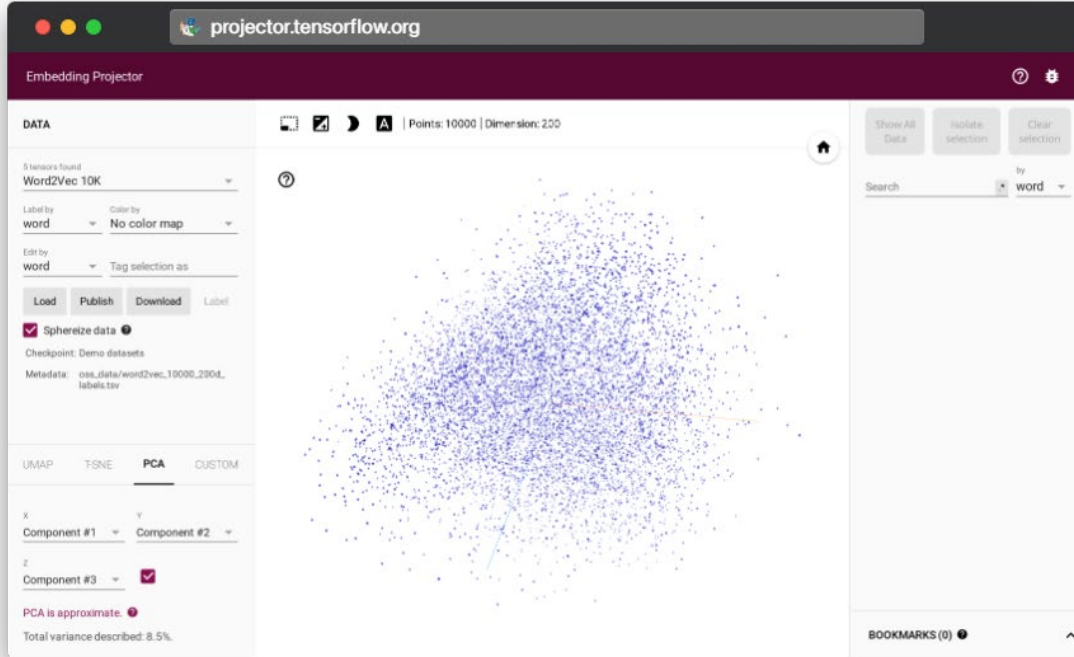


queen



Quelle: [The Illustrated Word2vec – Jay Alammar – Visualizing machine learning one concept at a time. \(jalammar.github.io\)](https://alammar.github.io/illustrated-word2vec/)

Word2vec Demo



projector.tensorflow.org

Fazit

- Zusammenspiel Tokenizer & Embeddings
 - Tokenizer: Zerlegen Text für Verarbeitung.
 - Word Embeddings: Transformieren Token in numerische Repräsentationen für Modelle.
 - Bedeutung in NLP: Grundlage moderner Modelle (z. B. Transformer, BERT, GPT).
- Takeaways
 - Tokenizer entscheiden, wie Text verarbeitet wird.
 - Embeddings sind essenziell, um semantisches Wissen in maschinenlesbare Form zu bringen.
 - Fortschrittliche Ansätze kombinieren Subword-Tokenization mit dichten Embeddings.